

APOSTILA · ARDUINO UNO

Guia de Apoio

Projetos com Arduino Uno



Material de apoio para a montagem e o entendimento dos projetos.

Use em conjunto com o repositório:

github.com/Gui1-p/Projetos-Arduino-Uno

Atualizado em 28/05/2026

Este material reúne tudo o que você precisa para entender e montar os projetos. Leia as seções 1 a 5 uma vez com calma; depois use a seção 6 como consulta rápida durante a montagem de cada projeto.

Repositório dos projetos

Todos os códigos (`.ino`) estão no GitHub, organizados em uma pasta por projeto:

<https://github.com/Guil-p/Projetos-Arduino-Uno>

Como usar o repositório:

1. Acesse o link acima e baixe os arquivos (botão verde **Code** → **Download ZIP**) ou clone com `git clone https://github.com/Guil-p/Projetos-Arduino-Uno.git`.
2. Abra a pasta do projeto desejado e dê dois cliques no arquivo `.ino` para abri-lo na IDE do Arduino.
3. **Confira sempre os `#define` no topo do código** — eles indicam em quais pinos ligar cada componente.
4. Conecte a placa, selecione a porta em *Ferramentas* → *Porta* e faça o upload (`Ctrl + U`).

Cada pasta tem também um arquivo `README.md` com a descrição do projeto.

1. O que é o Arduino Uno?

O Arduino Uno é uma plataforma de computação física de código aberto, formada por um microcontrolador simples e um ambiente de desenvolvimento (a IDE do Arduino). Ele é o "cérebro" dos projetos: lê informações de sensores (entradas) e controla dispositivos físicos (saídas), como LEDs e buzzers.

Características principais:

- **Microcontrolador ATmega328** — processa o programa e o mantém na memória.
- **14 pinos digitais** — operam só com 0 V (LOW) ou 5 V (HIGH). Dentre eles, **6 pinos (3, 5, 6, 9, 10, 11)** têm o símbolo  e conseguem simular saída analógica usando PWM.
- **6 entradas analógicas (A0 a A5)** — leem tensões variáveis entre 0 V e 5 V, úteis para sensores cujo valor não é apenas "ligado/desligado".
- **Pinos de alimentação** — `5V` e `3.3V` fornecem energia para os módulos; os pinos `GND` são o "negativo" / referência comum (existem três deles na placa).
- **Estrutura do programa** — todo código tem `setup()`, que roda **uma vez** na inicialização, e `loop()`, que se repete **infinitamente** depois disso.

2. Conceitos de eletrônica para os projetos

Antes de programar, é preciso ligar os componentes corretamente. Esta seção evita os erros de montagem mais comuns.

Alimentação: VCC e GND

Quase todo módulo tem três pinos: **VCC** (energia, ligar no **5V** do Arduino), **GND** (terra, ligar em qualquer **GND** do Arduino) e **um pino de sinal** (S, DO, OUT, A0, etc.), que é por onde a informação passa. **Inverter VCC e GND pode danificar o módulo** — confira sempre antes de ligar a placa no USB.

Regra de ouro: todos os GND do circuito (Arduino, módulos, protoboard) precisam estar interligados. Sem um "negativo comum", os sinais não funcionam.

Protoboard e jumpers

A protoboard conecta componentes sem solda. As **duas linhas laterais** (marcadas com **+** e **-**) são trilhas contínuas, usadas para distribuir 5 V e GND. As **colunas centrais** são ligadas em grupos de 5 furos, separadas por um canal no meio. Os **jumpers** são apenas fios para fazer essas ligações.

Sinal digital × sinal analógico

- **Digital**: só dois valores possíveis — **HIGH** (5 V, "1") ou **LOW** (0 V, "0"). Um botão, por exemplo, é digital.
- **Analógico**: uma faixa contínua de valores. O Arduino lê isso com `analogRead`, devolvendo um número de **0 a 1023**.

PWM (saída analógica simulada)

O Arduino não consegue gerar uma tensão "intermediária" de verdade. Em vez disso, ele liga e desliga o pino muito rápido (PWM). Quanto mais tempo ligado, maior o "brilho médio". Usamos `analogWrite` com valores de **0 a 255** para isso. Só funciona nos 6 pinos PWM (marcados com **~**).

Lógica ativo-alto e ativo-baixo

Atenção a um detalhe que confunde muita gente: **muitos sensores e botões enviam LOW quando acionados**, não **HIGH**. Isso se chama lógica *ativo-baixo*. Por isso alguns códigos verificam `if (digitalRead(pino) == LOW)` para detectar um disparo. Se um projeto agir "ao contrário", quase sempre é só inverter LOW/HIGH no `if`.

Resistor de pull-up

Um pino de entrada "solto" (sem nada definido) capta ruído elétrico e gera leituras falsas. O pull-up resolve isso mantendo o pino em `HIGH` por padrão; quando o botão é apertado, ele cai para `LOW`. O Arduino tem pull-ups internos: basta usar `pinMode(pino, INPUT_PULLUP)` e nenhum resistor físico é necessário.

3. Estrutura de um programa Arduino

setup() e loop()

```
void setup() {  
    // roda UMA vez: configurar pinos, iniciar Serial, iniciar sensores  
}  
  
void loop() {  
    // roda SEM PARAR: ler sensores, decidir, agir  
}
```

Uma variável declarada dentro do `setup()` **não pode** ser usada no `loop()` — são blocos separados. Para guardar informação entre uma repetição e outra do `loop()`, a variável precisa ser **global** (declarada fora das funções, no topo do arquivo).

Variáveis e seus tipos

Tipo	Guarda	Exemplo
<code>int</code>	número inteiro (−32768 a 32767)	<code>int leitura = 512;</code>
<code>float</code>	número com casas decimais	<code>float temp = 24.7;</code>
<code>bool</code>	apenas <code>true</code> ou <code>false</code>	<code>bool ligado = false;</code>
<code>unsigned long</code>	número inteiro muito grande, sem sinal	usado com <code>millis()</code>
<code>const</code>	valor que nunca muda depois de definido	<code>const int debounce=50;</code>

#define e #include

- `#define NOME valor` — cria um "apelido". Antes de compilar, o nome é trocado pelo valor em todo o código. Usado para dar nome aos pinos: `#define LED 12`. Não leva ponto e vírgula no final.

- `#include <Biblioteca.h>` — adiciona código pronto de uma biblioteca (como a do sensor DHT11). Bibliotecas se instalam pela IDE em *Sketch* → *Incluir Biblioteca* → *Gerenciar Bibliotecas*.

Comentários

`// comentário de uma linha` ou `/* comentário de várias linhas */`. São ignorados pelo Arduino e servem só para humanos lerem.

Arrays (vetores)

Um array guarda **vários valores do mesmo tipo** em uma única variável, acessados por um índice que começa em **0**:

```
int notas[] = {330, 440, 466}; // notas[0] vale 330, notas[1] vale 440 ...
```

`sizeof(notas) / sizeof(notas[0])` calcula automaticamente quantos itens existem no array — útil para percorrê-lo com um `for` sem contar na mão.

Controle de fluxo

```
if (condicao) { ... } else { ... } // executa um bloco OU outro
```

```
for (int i = 0; i < 10; i++) { ... } // repete o bloco 10 vezes
```

Operadores usados nos projetos: `==` (igual), `!=` (diferente), `>` `<` (maior / menor), `!` (não / inverte), `||` (ou), `&&` (e).

4. Componentes usados nos projetos

Componentes de saída (agem sobre o mundo)

LED e LED RGB. Um LED acende quando recebe corrente no sentido certo. O LED RGB é, na prática, três LEDs (vermelho, verde, azul) no mesmo encapsulamento; variando o brilho de cada canal com PWM, obtêm-se outras cores. LEDs sempre precisam de um resistor em série para limitar a corrente — nos **módulos** prontos esse resistor já vem embutido na plaquinha.

Buzzer. Emite som quando recebe pulsos elétricos. Existem dois tipos, e entender a diferença ajuda a interpretar o resultado:

- **Buzzer passivo** — não tem oscilador próprio, então **você controla a frequência** pelo código. É o tipo que reproduz as notas certas (Dó, Ré, Mi...) e toca a melodia com clareza. É o ideal para os projetos `Buzzer_Batida` e `Buzzer_foto_interruptor`.

- **Buzzer ativo** — tem oscilador interno e um tom fixo. Mandar um sinal de áudio para ele (o `tone()` ou os pulsos do código) liga e desliga esse oscilador no ritmo do sinal — então **sai som e o ritmo da melodia aparece**, mas todas as notas saem mais ou menos no mesmo tom, com uma sonoridade mais "zumbida". O projeto funciona; só não se ouve a melodia de verdade.

Os dois buzzers funcionam nesses projetos — a diferença é de fidelidade. Vale testar o mesmo código nos dois e ouvir a diferença: é uma boa forma de entender na prática o que é controlar a frequência de um som.

Componentes de entrada (sensores)

Botão (pushbutton). Fecha o circuito enquanto pressionado. Combinado com `INPUT_PULLUP`, lê `HIGH` solto e `LOW` apertado.

Joystick. São dois potenciômetros (resistores variáveis) montados juntos. Cada eixo (VRx, VRy) é lido como valor analógico (0–1023). Tem ainda um botão digital embutido (pino SW).

Encoder rotativo. Um seletor giratório sem fim. Ao girar, seus contatos geram uma sequência de pulsos nos pinos **CLK** e **DT**. Comparando esses dois pinos é possível descobrir o **sentido** do giro (horário ou anti-horário) e contar "passos". Costuma ter também um botão (SW) acionado ao pressionar o eixo. Atenção: o encoder **não** é um simples interruptor liga/desliga — ele informa *movimento*, não um estado fixo.

Sensor de som digital. Um microfone com um circuito que envia `HIGH` (ou `LOW`, depende do módulo) quando o ruído passa de um limite ajustável por um potenciômetro/trimpot na plaquinha.

Sensor foto-interruptor. Um emissor e um receptor de luz infravermelha frente a frente, com uma fenda entre eles. Quando um objeto entra na fenda e bloqueia a luz, o pino de sinal muda de estado.

Sensor de temperatura e umidade DHT11. Sensor digital que mede temperatura (em °C) e umidade do ar e entrega os dois valores ao Arduino através de uma biblioteca (a da Adafruit). É um sensor lento: precisa de um intervalo de pelo menos 2 segundos entre leituras.

5. Funções de programação

5.1 Configuração de pinos

`pinMode(pino, modo)` Define se um pino é entrada ou saída. Parâmetros: `pino` (número da porta); `modo` = `INPUT`, `OUTPUT` ou `INPUT_PULLUP`. Retorno: nenhum.

5.2 Entrada e saída digital

digitalWrite(pino, valor) Envia **HIGH** (5 V) ou **LOW** (0 V) a um pino de saída — acende um LED, aciona um relé, etc. Parâmetros: **pino**; **valor** = **HIGH** ou **LOW**. Retorno: nenhum.

digitalRead(pino) Lê o estado de um pino digital de entrada. Parâmetro: **pino**. Retorno: **int** valendo **HIGH** (1) ou **LOW** (0).

5.3 Entrada e saída analógica (e PWM)

analogRead(pino) Lê a tensão de um pino analógico. Parâmetro: **pino** (A0 a A5). Retorno: **int** de **0 a 1023**.

analogWrite(pino, valor) Gera um sinal PWM para simular saída analógica (controlar brilho, por exemplo). Parâmetros: **pino** (precisa ser um pino PWM, com **~**); **valor** de **0 a 255**. Retorno: nenhum.

Cuidado clássico: **analogRead** vai até 1023, mas **analogWrite** só aceita até 255. Para ligar um diretamente no outro, **converta a escala** com **map()** (ver 5.6). Sem isso, valores acima de 255 "dão a volta" e o brilho fica errado.

5.4 Tempo

delay(tempo) Congela o processador pelo tempo indicado, em milissegundos. Nada mais acontece durante esse período. Parâmetro: **tempo** (ms). Retorno: nenhum.

delayMicroseconds(tempo) Igual ao **delay**, mas em microssegundos (1 ms = 1000 µs). Usado para gerar tempos muito curtos, como a onda sonora "manual" de um buzzer passivo. Parâmetro: **tempo** (µs). Retorno: nenhum.

millis() Funciona como um cronômetro: devolve há quantos milissegundos a placa está ligada. Diferente do **delay()**, **não congela** o programa — por isso é usada para medir tempo (debounce, cooldown) sem travar o resto do código. Parâmetros: nenhum. Retorno: **unsigned long**.

5.5 Som

tone(pino, frequencia) / **tone(pino, frequencia, duracao)** Gera uma onda quadrada na frequência indicada, em hertz. Num **buzzer passivo** isso toca a nota correta; num buzzer ativo sai som no ritmo certo, mas sem a afinação. Com o terceiro parâmetro, a nota para sozinha após a duração dada (em ms). O som toca "em segundo plano": o programa continua executando. Retorno: nenhum.

noTone(pino) Interrompe o som iniciado por **tone()** naquele pino. Retorno: nenhum.

5.6 Comunicação serial (mostrar dados no computador)

`Serial.begin(velocidade)` Inicia a comunicação com o computador, no `setup()`. A velocidade (geralmente `9600`) **precisa ser a mesma** selecionada no Monitor Serial.

`Serial.print(dado)` / `Serial.println(dado)` Envia um número, variável ou `"texto"` para o Monitor Serial. O `println` pula para a linha seguinte ao final. Retorno: número de bytes escritos (em geral ignorado).

Para abrir o Monitor Serial: ícone de lupa no canto superior direito da IDE, ou *Ferramentas → Monitor Serial*. Confira a velocidade (baud) no canto da janela.

5.7 Funções auxiliares úteis

`map(valor, minOrigem, maxOrigem, minDestino, maxDestino)` Converte um número de uma escala para outra, proporcionalmente. É a solução para o problema do 0–1023 → 0–255:

```
analogWrite(led, map(analogRead(A0), 0, 1023, 0, 255));
```

`constrain(valor, minimo, maximo)` Garante que um valor não saia de um intervalo: se passar do máximo, vira o máximo; se ficar abaixo do mínimo, vira o mínimo.

`random(minimo, maximo)` Gera um número pseudoaleatório (inclui o mínimo, exclui o máximo). `random(1, 7)` sorteia de 1 a 6 — útil para um dado eletrônico.

`isnan(valor)` "Is Not a Number". Devolve `true` se um valor não for um número válido. Usada para detectar falha de leitura de um sensor (ex.: DHT11 com fio solto).

`sizeof(variavel)` Devolve o tamanho, em bytes, de uma variável. Combinada como `sizeof(array)/sizeof(array[0])`, conta quantos itens há num array.

`return` Encerra a função imediatamente. Dentro do `loop()`, faz pular o resto da repetição atual e recomeçar do início — útil para abortar um ciclo quando a leitura falhou.

6. Consulta rápida por projeto

Os pinos abaixo seguem o código atual do repositório; sempre confira os `#define` no topo de cada `.ino`.

Led_Botao

- **O que faz:** um botão liga/desliga um LED, alternando a cada toque.
- **Componentes:** módulo botão, módulo LED.
- **Pinos:** botão = 13 (`INPUT_PULLUP`); LED = 12.
- **Conceitos-chave:** `digitalRead` / `digitalWrite`, detecção de *borda* (agir só no instante em que aperta), debounce com `millis()`.

Led_joystick

- **O que faz:** os eixos do joystick controlam o brilho dos canais vermelho e verde; o botão do joystick liga/desliga o azul.
- **Componentes:** módulo joystick, módulo LED RGB.
- **Pinos:** VRx = A0; VRy = A1; SW = 8 (`INPUT_PULLUP`); LEDs = 11, 10, 9.
- **Conceitos-chave:** `analogRead` , `analogWrite` (PWM), conversão de escala com `map()` , leitura de borda no botão.

Led_Encoder

- **O que faz:** seletor de cores — o botão escolhe o canal (R, G ou B) e o giro ajusta esse canal.
- **Componentes:** módulo encoder rotativo, módulo LED RGB.
- **Pinos:** LEDs = 11, 10, 9; botão SW = 7 (`INPUT_PULLUP`); CLK = 6.
- **Conceitos-chave:** leitura de encoder, variáveis para guardar estado, uso de `#define` como apelido para números.

Led_Sensor_Som

- **O que faz:** alterna o LED a cada som detectado (uma palma, por exemplo).
- **Componentes:** módulo sensor de som digital, módulo LED.
- **Pinos:** sensor = 2; LED = 13.
- **Conceitos-chave:** sensor digital, ajuste de sensibilidade no trimpot do módulo, controle de tempo para não disparar várias vezes seguidas.

Buzzer_Batida

- **O que faz:** ao ser acionado, o buzzer toca uma sequência de notas (alarme musical).
- **Componentes:** buzzer (passivo de preferência, para ouvir a melodia; o ativo também funciona), um sensor digital de disparo.
- **Pinos:** buzzer = 13; sensor = 8 (`INPUT_PULLUP` , dispara em `LOW`).
- **Conceitos-chave:** geração de som com `delayMicroseconds` , relação entre frequência e período, lógica ativo-baixo.

Buzzer_foto_interruptor

- **O que faz:** quando um objeto passa pelo foto-interruptor, toca uma melodia.
- **Componentes:** buzzer (passivo de preferência, para ouvir a melodia; o ativo também funciona), módulo sensor foto-interruptor.
- **Pinos:** buzzer = 8; sensor = 2.
- **Conceitos-chave:** `tone()` / `noTone()` , arrays para guardar a melodia, `for` para percorrer o array, `sizeof` .

Sensor_temp_umidade

- **O que faz:** lê temperatura e umidade e mostra no Monitor Serial.
 - **Componentes:** módulo DHT11.
 - **Pinos:** dado do DHT11 = 13. **Requer a biblioteca "DHT sensor library" da Adafruit** (instale também as dependências sugeridas, como a Adafruit Unified Sensor).
 - **Conceitos-chave:** `#include` e bibliotecas, comunicação serial, verificação de erro com `isnan()`, intervalo mínimo de 2 s entre leituras.
-

7. Erros comuns e como evitar

- **LED ou módulo não funciona:** confira se VCC e GND não estão invertidos e se todos os GND estão interligados.
 - **Brilho do LED errado no joystick:** faltou converter 0-1023 para 0-255 com `map()`.
 - **Botão "pula" valores ou pisca sozinho:** falta detecção de borda — só aja quando o estado *mudar* de solto para apertado, e guarde a leitura anterior numa variável.
 - **Sensor age ao contrário:** inverta `LOW / HIGH` no `if`; muitos módulos são ativo-baixo.
 - **A melodia sai "zumbida" ou sem afinação:** está sendo usado o buzzer ativo — ele toca o ritmo certo, mas num tom fixo. Para ouvir as notas da melodia, use o buzzer passivo.
 - **Monitor Serial mostra símbolos estranhos:** a velocidade (baud) da janela está diferente da definida em `Serial.begin()`.
 - **DHT11 retorna "Falha na leitura":** sensor é lento — respeite os 2 s entre leituras — e confira o pino de dados.
 - **O programa "trava" enquanto algo acontece:** provavelmente há um `delay()` longo; para manter o Arduino responsivo, prefira medir tempo com `millis()`.
-

8. Onde pesquisar mais

Projeto e ferramentas

- **Repositório dos projetos (códigos `.ino`):** <https://github.com/Gui1-p/Projetos-Arduino-Uno>
- **IDE do Arduino (download):** <https://www.arduino.cc/en/software>

- **Tinkercad Circuits** — simulador online gratuito, monta o circuito e roda o código no navegador, sem precisar de placa física: <https://www.tinkercad.com/circuits>
- **Wokwi** — outro simulador online, útil para testar a lógica do código: <https://wokwi.com>

Documentação oficial do Arduino

- **Referência da linguagem (em português)** — explica cada função, com exemplos. Consulte aqui qualquer comando que aparecer no código: <https://www.arduino.cc/reference/pt/>
- **Documentação geral do Arduino:** <https://docs.arduino.cc>
- **Página do Arduino Uno** — pinagem, características da placa: <https://docs.arduino.cc/hardware/uno-rev3>

Bibliotecas usadas nos projetos

- **DHT sensor library (Adafruit)** — biblioteca do sensor de temperatura e umidade DHT11. Instale pela IDE em *Sketch* → *Incluir Biblioteca* → *Gerenciar Bibliotecas*, buscando por "DHT sensor library": <https://github.com/adafruit/DHT-sensor-library>
- **Guia da Adafruit sobre o DHT** — explica ligação e uso do sensor: <https://learn.adafruit.com/dht>

Material de apoio para aprender e tirar dúvidas

- **Arduino Project Hub** — projetos prontos com esquema e código, bom para inspiração e comparação: <https://projecthub.arduino.cc>
- **Fórum oficial do Arduino** — para perguntar quando algo não funciona; tem seção em português: <https://forum.arduino.cc>
- **Random Nerd Tutorials** — tutoriais detalhados de sensores e módulos: <https://randomnerdtutorials.com>
- **Last Minute Engineers** — explicações visuais de como cada módulo funciona e como ligá-lo: <https://lastminuteengineers.com>

Dica de pesquisa

Ao procurar como usar um módulo específico, pesquise pelo **nome do módulo + a sigla "KY"** quando houver (ex.: "encoder KY-040", "joystick KY-023"). A maioria dos módulos didáticos pertence à série **KY**, e há documentação farta sobre cada um com esse padrão de nome.